



ESCUELA POLITÉCNICA NACIONAL

TEMA: Proyecto

Final

MATERIA:

Verificación y Validación del software

INTEGRANTES:

Javier Andres Angulo Albornoz

Priscila Salome Pazmiño Santacruz

Cladio Omar Peñaherrera Llulluna

Saul Gonzalo Tualombo Benavides

Carol Abigail Velasquez Cando

FECHA

16/07/2026



Contenido

1.	Introducción	3
2.	Objetivos	3
2.1	Objetivo general	3
2.2	Objetivos específicos	3
3.	Herramientas utilizadas	3
4.	Metodología de trabajo	3
5.	Análisis inicial de los controladores	4
5.1	Estructura del Backend	4
5.1.1	CitaController	4
5.1.2	DoctorController	4
5.1.3	HistoriaClinicaController	5
5.1.4	PacienteController	5
5.2	Particiones de equivalencia	6
5.2.1	CitaController	6
5.2.2	DoctorController	6
5.2.3	HistoriaClinicaController	6
5.2.4	PacienteController	7
5.3	Pruebas de integración aplicadas	7
5.3.1	CitasControllerIntegrationTest	7
5.3.2	Bugs / Defectos encontrados	8
5.3.3	DoctorControllerIntegrationTest	9
5.3.4	Bugs / Defectos encontrados	10
5.3.5	HistoriaClinicaControllerIntegrationTest	11
5.3.6	Bugs / Defectos encontrados	12
5.3.7	PacienteControllerIntegrationTest	13
5.3.8	Bugs / Defectos encontrados	14
6.	Conclusiones	16



1. Introducción

La verificación de calidad de un sistema no se limita a comprobar que el código compile o que la aplicación funcione en apariencia, sino que también implica confirmar que cada componente se comporta correctamente al integrarse con los demás y que los flujos completos de negocio responden como se espera, incluso ante entradas maliciosas o mal formadas.

En este trabajo se realizaron las pruebas de integración del backend del proyecto Hospital Management, desarrollado con Spring Boot, utilizando `@SpringBootTest` en conjunto con `MockMvc` para levantar el contexto completo de la aplicación y simular peticiones HTTP reales sobre los distintos endpoints, verificando tanto el código de estado devuelto como el contenido de las respuestas y evaluando la interacción real entre controladores, servicios, capa de persistencia y validaciones.

Durante la ejecución se documentaron de forma intencional varios defectos presentes en el sistema, entre ellos vulnerabilidades de inyección SQL, XSS almacenado, códigos de estado HTTP incorrectos ante escenarios de error y ausencia de validaciones en ciertos endpoints, registrando cada hallazgo junto con la prueba que lo evidencia, sin modificar el código base proporcionado.

2. Objetivos

2.1 Objetivo general

- Verificar el comportamiento funcional y de seguridad del backend del proyecto Hospital Management mediante pruebas de integración con `@SpringBootTest` y `MockMvc`, para comprobar el correcto funcionamiento conjunto de los componentes del sistema y documentar los defectos existentes en sus endpoints.

2.2 Objetivos específicos

- Simular peticiones HTTP reales sobre los distintos endpoints del sistema mediante `MockMvc`, para evaluar la interacción entre controladores, servicios y capa de persistencia.
- Evidenciar vulnerabilidades y errores presentes en el backend, tales como inyección SQL, XSS almacenado, códigos de estado incorrectos y validaciones ausentes, mediante pruebas de integración documentadas, para dejar constancia del comportamiento real del sistema sin modificar el código base.

3. Herramientas utilizadas

- **IntelliJ IDEA:** entorno de desarrollo utilizado para revisar y corregir el backend.

4. Metodología de trabajo

- Se clonó el repositorio del proyecto en el equipo local.
- Se revisó cada uno de los controladores para comprender su funcionalidad, información que además se encontraba especificada en el README del proyecto.
- Se realizó el particionamiento de los datos de entrada para cada función de los controladores, con el fin de definir los casos de prueba a cubrir.
- Con base en dicho particionamiento, se redactaron las pruebas de integración correspondientes a cada controlador
- Finalmente, se ejecutaron y se observaron los resultados obtenidos.



5. Análisis inicial de los controladores

5.1 Estructura del Backend

El backend del proyecto **Hospital Management** es una API REST desarrollada para gestionar los procesos principales de un sistema hospitalario. Su arquitectura se encuentra organizada mediante el uso de controladores, donde cada uno se encarga de administrar una entidad específica del dominio, como pacientes, doctores, citas médicas e historias clínicas.

Durante el análisis del backend también se identificaron algunos problemas de implementación relacionados con las buenas prácticas en APIs REST que se detallaran a continuación.

5.1.1 CitaController

Este controlador administra todas las operaciones relacionadas con las citas médicas del sistema, permitiendo registrar, consultar, actualizar y eliminar citas, además de realizar búsquedas por paciente, doctor, estado y rango de fechas.

Método	Endpoint	Descripción	Observación
GET	/api/citas	Lista todas las citas registradas.	—
GET	/api/citas/{id}	Obtiene una cita mediante su identificador.	—
POST	/api/citas	Registra una nueva cita médica.	Retorna 200 OK en lugar de 201 Created.
PUT	/api/citas/{id}	Actualiza la información de una cita existente.	—
DELETE	/api/citas/{id}	Elimina una cita del sistema.	—
GET	/api/citas/paciente/{pacienteId}	Lista las citas pertenecientes a un paciente.	—
GET	/api/citas/doctor/{doctorId}	Lista las citas asignadas a un doctor.	—
GET	/api/citas/estado/{estado}	Obtiene las citas filtradas por estado.	—
GET	/api/citas/rango-fechas	Obtiene las citas comprendidas entre una fecha de inicio y una fecha de fin.	—

Tabla 1. Endpoints del módulo Cita.

5.1.2 DoctorController

Este controlador gestiona la información de los doctores registrados en el sistema, permitiendo operaciones CRUD y consultas específicas por especialidad o nombre.

Método	Endpoint	Descripción	Observación
GET	/api/doctores	Lista todos los doctores registrados.	—
GET	/api/doctores/{id}	Obtiene un doctor por su identificador.	—
POST	/api/doctores	Registra un nuevo doctor.	Retorna 200 OK en lugar de 201 Created.



PUT	/api/doctores/{id}	Actualiza la información de un doctor existente.	—
DELETE	/api/doctores/{id}	Elimina un doctor.	—
GET	/api/doctores/buscar-especialidad	Busca doctores según su especialidad mediante el parámetro q.	Posible SQL Injection, ya que el parámetro q se utiliza de forma insegura en la consulta.
GET	/api/doctores/buscar-nombre	Busca doctores por nombre y apellido.	—

Tabla 2.Endpoints del módulo Doctor.

5.1.3 HistoriaClinicaController

Este controlador administra las historias clínicas de los pacientes. A diferencia de los demás controladores, únicamente permite registrar y consultar información, sin ofrecer operaciones de actualización o eliminación.

Método	Endpoint	Descripción	Observación
GET	/api/historias-clinicas	Lista todas las historias clínicas registradas.	—
GET	/api/historias-clinicas/{id}	Obtiene una historia clínica por su identificador.	—
POST	/api/historias-clinicas	Registra una nueva historia clínica.	⚠ Retorna 200 OK en lugar de 201 Created.
GET	/api/historias-clinicas/paciente/{pacienteId}	Lista las historias clínicas de un paciente.	—
GET	/api/historias-clinicas/doctor/{doctorId}	Lista las historias clínicas registradas por un doctor.	—

Tabla 3.Endpoints del módulo Historia Clínica.

5.1.4 PacienteController

Este controlador administra la información de los pacientes, permitiendo realizar operaciones CRUD y consultas relacionadas con búsquedas y estadísticas de edad.

Método	Endpoint	Descripción	Observación
GET	/api/pacientes	Lista todos los pacientes registrados.	—
GET	/api/pacientes/{id}	Obtiene un paciente por su identificador.	—
POST	/api/pacientes	Registra un nuevo paciente.	⚠ Retorna 200 OK en lugar de 201 Created.
PUT	/api/pacientes/{id}	Actualiza la información de un paciente existente.	—
DELETE	/api/pacientes/{id}	Elimina un paciente.	⚠ Retorna 200 OK en lugar de 204 No Content.
GET	/api/pacientes/buscar	Busca pacientes por nombre.	—
GET	/api/pacientes/estadisticas/edad-promedio	Calcula la edad promedio de los pacientes registrados.	—



Tabla 4. Endpoints del módulo Paciente.

5.2 Particiones de equivalencia

5.2.1 CitaController

Endpoint	Parámetro	Partición válida	Partición inválida
POST /api/citas	pacienteId	ID existente en BD	ID inexistente, nulo, negativo
	doctorId	ID existente en BD	ID inexistente, nulo, negativo
	fechaHora	Fecha futura	Fecha pasada, nula, formato incorrecto
	motivo	Texto no vacío, dentro de longitud permitida	Vacío, nulo, excede longitud máxima
GET /api/citas/{id}	id	ID existente	ID inexistente, no numérico, negativo
PUT /api/citas/{id}	id + dto	Igual que POST + id existente	id inexistente, mismos casos inválidos del DTO
DELETE /api/citas/{id}	id	ID existente	ID inexistente, no numérico
GET /api/citas/paciente/{pacienteId}	pacienteId	ID con citas asociadas	ID sin citas, inexistente
GET /api/citas/doctor/{doctorId}	doctorId	ID con citas asociadas	ID sin citas, inexistente
GET /api/citas/estado/{estado}	estado	Valor válido: "PROGRAMADA", "COMPLETADA", "CANCELADA"	Valor no reconocido, vacío, mal escrito
GET /api/citas/rango-fechas	inicio, fin	inicio < fin, ambas fechas válidas	inicio > fin, fechas iguales, formato inválido, parámetro faltante

Tabla 5. Particiones de equivalencia del controller de citas.

5.2.2 DoctorController

Endpoint	Parámetro	Partición válida	Partición inválida
POST /api/doctores	nombre/apellido	Texto no vacío, solo letras	Vacío, nulo, con caracteres especiales/números
	especialidad	Valor de catálogo existente	Vacío, nulo, no reconocido
GET /api/doctores/{id}	id	ID existente	ID inexistente, no numérico
PUT /api/doctores/{id}	id + dto	Igual que POST + id existente	id inexistente, mismos casos inválidos del DTO
DELETE /api/doctores/{id}	id	ID existente	ID inexistente
GET /api/doctores/buscar-especialidad	q	Texto plano (nombre de especialidad)	Cadena con SQL embebido (' OR '1'='1'), vacío, caracteres especiales. Relevante para el bug de SQL Injection.
GET /api/doctores/buscar-nombre	nombre, apellido	Ambos con texto válido y coincidencia en BD	Uno o ambos vacíos, sin coincidencia, con espacios extra

Tabla 6. Particiones de equivalencia del controller de Doctor.

5.2.3 HistoriaClinicaController

Endpoint	Parámetro	Partición válida	Partición inválida
POST /api/historias-clinicas	pacienteId	ID existente	ID inexistente, nulo
	doctorId	ID existente	ID inexistente, nulo
	descripcion/diagnóstico	Texto no vacío dentro de longitud permitida	Vacío, nulo, excede longitud, con HTML/script embebido.
GET /api/historias-clinicas/{id}	id	ID existente	ID inexistente, no numérico
GET /api/historias-clinicas/paciente/{pacienteId}	pacienteId	ID con historias asociadas	ID sin historias, inexistente



GET /api/historias-clinicas/doctor/{doctorId}	doctorId	ID con historias asociadas	ID sin historias, inexistente
--	----------	----------------------------	-------------------------------

Tabla 7. Particiones de equivalencia del controller de Historia Clínica.

5.2.4 PacienteController

Endpoint	Parámetro	Partición válida	Partición inválida
POST /api/pacientes	nombre/apellido	Texto no vacío, solo letras	Vacío, nulo, con números/caracteres especiales
	fechaNacimiento	Fecha pasada, edad razonable	Fecha futura, fuera de rango (ej. edad negativa o >120 años)
	email	Formato válido (x@x.com)	Formato inválido, vacío, duplicado en BD
GET /api/pacientes/{id}	id	ID existente	ID inexistente, no numérico
PUT /api/pacientes/{id}	id + dto	Igual que POST + id existente	id inexistente, mismos casos inválidos del DTO
DELETE /api/pacientes/{id}	id	ID existente	ID inexistente
GET /api/pacientes/buscar	nombre	Nombre con coincidencia parcial o total en BD	Vacío, sin coincidencia, con caracteres especiales
GET /api/pacientes/estadisticas/edad-promedio	— (sin parámetros)	BD con al menos un paciente	BD vacía (posible división por cero o resultado nulo)

Tabla 8. Particiones de equivalencia del controller de Paciente

5.3 Pruebas de integración aplicadas

5.3.1 CitasControllerIntegrationTest

Se realizaron 37 pruebas de integración:

#	Test	Endpoint	Partición / Caso	Resultado esperado
1	crear_datosValidos_retorna200	POST /api/citas	Datos válidos	201
2	crear_doctorInexistente_retorna404	POST /api/citas	doctorId inexistente	404
3	crear_pacienteIdNulo_retorna400	POST /api/citas	pacienteId nulo	400
4	crear_fechaPasada_retorna400	POST /api/citas	Fecha pasada	400
5	crear_fechaActual_retorna201	POST /api/citas	Fecha actual (valor límite)	201, se acepta con @FutureOrPresent
6	crear_pacienteInexistente_retorna404	POST /api/citas	pacienteId inexistente	404
7	crear_dobleBookingMismoDoctor_retorna400	POST /api/citas	Doble reserva mismo doctor/hora	201 la primera, 400 la segunda
8	crear_doctorIdNulo_retorna400	POST /api/citas	doctorId nulo	400
9	crear_pacienteIdNegativo_retorna400	POST /api/citas	pacienteId negativo	400
10	crear_motivoVacio_retorna400	POST /api/citas	motivo vacío	400
11	crear_motivoNulo_retorna400	POST /api/citas	motivo nulo	400
12	crear_motivoExcedeLongitud_retorna400	POST /api/citas	motivo excede longitud máxima	400
13	crear_fechaHoraNula_retorna400	POST /api/citas	fechaHora nula	400



Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas
“Verificación y Validación del Software”

14	buscar_idExistente_retornaCita	GET /api/citas/{id}	ID existente	200 con la cita
15	buscar_idInexistente_retorna404	GET /api/citas/{id}	ID inexistente	404
16	buscar_idNoNumerico_retorna400	GET /api/citas/{id}	ID no numérico ("abc")	400
17	buscar_idNegativo_retorna404	GET /api/citas/{id}	ID negativo	404
18	actualizar_datosValidos_retornaActualizada	PUT /api/citas/{id}	Actualización válida	200 con datos actualizados
19	actualizar_idInexistente_retorna404	PUT /api/citas/{id}	ID inexistente	404
20	actualizar_pacienteIdNulo_retorna400	PUT /api/citas/{id}	pacienteId nulo	400
21	actualizar_fechaPasada_retorna400	PUT /api/citas/{id}	Fecha pasada	400
22	eliminar_idExistente_retorna204	DELETE /api/citas/{id}	ID existente	204, registro eliminado
23	eliminar_idInexistente_retorna404	DELETE /api/citas/{id}	ID inexistente	404
24	listarPorPaciente_conCitas_retornaLista	GET /api/citas/paciente/{id}	Paciente con citas	200, lista tamaño 1
25	listarPorPaciente_sinCitas_retornaListaVacía	GET /api/citas/paciente/{id}	Paciente existente sin citas	200, lista vacía
26	listarPorPaciente_pacienteInexistente_retornaListaVacía	GET /api/citas/paciente/{id}	Paciente inexistente	200, lista vacía
27	listarPorDoctor_conCitas_retornaLista	GET /api/citas/doctor/{id}	Doctor con citas	200, lista tamaño 1
28	listarPorDoctor_sinCitas_retornaListaVacía	GET /api/citas/doctor/{id}	Doctor existente sin citas	200, lista vacía
29	listarPorDoctor_doctorInexistente_retornaListaVacía	GET /api/citas/doctor/{id}	Doctor inexistente	200, lista vacía
30	listarPorEstado_estadoExistente_retornaListaOrdenada	GET /api/citas/estado/{estado}	Estado con citas (2)	200, lista ordenada por fecha
31	listarPorEstado_estadoNoReconocido_retornaListaVacía	GET /api/citas/estado/{estado}	Estado no reconocido ("INVENTADO")	200, lista vacía
32	listarPorEstado_sinCoincidencias_retornaListaVacía	GET /api/citas/estado/{estado}	Estado válido sin coincidencias	200, lista vacía
33	listarPorRangoFechas_rangoValido_retornaCoincidencias	GET /api/citas/rango-fechas	Rango válido con coincidencias	200, lista tamaño 1
34	listarPorRangoFechas_fechasIguales_retornaListaVacía	GET /api/citas/rango-fechas	inicio = fin (valor límite)	200, lista vacía
35	listarPorRangoFechas_formatoInvalido_retorna400	GET /api/citas/rango-fechas	Formato de fecha inválido	400
36	listarPorRangoFechas_parametroFaltante_retorna400	GET /api/citas/rango-fechas	Parámetro fin faltante	400
37	listarPorRangoFechas_inicioMayorQueFin_retorna400	GET /api/citas/rango-fechas	inicio > fin	400

Tabla 9, Listado de pruebas de integración del controlador de citas.

5.3.2 Bugs / Defectos encontrados

#	Problema detectado	Solución aplicada	Archivo modificado
1	DELETE /api/citas/{id} retornaba 200 OK en vez de 204 No Content.	Se reemplazó ResponseEntity.ok().build() por ResponseEntity.noContent().build().	CitaController.java
2	El método DELETE no verificaba si el ID existía antes de eliminar.	Se agregó una validación con existsById() en el servicio; si el registro no existe, se lanza ResourceNotFoundException antes de eliminar.	CitaService.java
3	El manejador de ResourceNotFoundException respondía con 200 OK en lugar de 404 Not Found.	Se cambió ResponseEntity.ok(body) por ResponseEntity.status(HttpStatus.NOT_FOUND).body(body).	GlobalExceptionHandler.java
4	El manejador de errores de validación exponía información interna mediante el campo debug.	Se eliminó el campo debug, dejando únicamente el mapa de errores por campo.	GlobalExceptionHandler.java



Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas
“Verificación y Validación del Software”

5	El manejador general devolvía el stack trace completo al cliente.	Se reemplazó por un mensaje genérico para evitar exposición de información sensible (<i>information leakage</i>).	GlobalExceptionHandler.java
6	No existía un manejador para IDs con formato inválido (por ejemplo, texto en lugar de número).	Se agregó un <code>@ExceptionHandler</code> para <code>MethodArgumentTypeMismatchException</code> , devolviendo 400 Bad Request.	GlobalExceptionHandler.java
7	No existía un manejador para parámetros obligatorios faltantes en la petición.	Se agregó un <code>@ExceptionHandler</code> para <code>MissingServletRequestParameterException</code> , retornando 400 Bad Request.	GlobalExceptionHandler.java
8	El campo motivo no tenía validaciones.	Se agregaron las anotaciones <code>@NotBlank</code> y <code>@Size(max = 255)</code> para validar contenido y longitud.	CitaDTO.java
9	La anotación <code>@Future</code> en fechaHora rechazaba la fecha actual.	Se reemplazó <code>@Future</code> por <code>@FutureOrPresent</code> , permitiendo citas en el momento actual.	CitaDTO.java
10	pacienteId y doctorId aceptaban valores negativos.	Se agregó la anotación <code>@Positive</code> a ambos campos, además de <code>@NotNull</code> .	CitaDTO.java
11	El rango de fechas no validaba que inicio fuera anterior a fin.	Se añadió una validación en el controlador que lanza <code>IllegalArgumentException</code> cuando <code>inicio > fin</code> , siendo gestionada por el manejador de excepciones existente.	CitaController.java
12	POST /api/citas retornaba 200 OK en vez de 201 Created.	Se reemplazó <code>ResponseEntity.ok()</code> por <code>ResponseEntity.created(uri)</code> , agregando el encabezado <code>Location</code> con la ubicación del recurso creado.	CitaController.java
13	Faltaba <code>@Transactional</code> en los métodos que modifican datos, sin rollback ante fallos.	Se agregó <code>@Transactional(readOnly = true)</code> a nivel de clase y <code>@Transactional</code> específico en crear, actualizar y eliminar.	CitaService.java
14	No se validaba que el paciente existiera al crear una cita.	Se agregó <code>pacienteRepository.existsById(...)</code> , lanzando <code>ResourceNotFoundException</code> si el paciente no existe.	CitaService.java (requiere inyectar <code>PacienteRepository</code>)
15	No se validaba conflicto de horario (doble booking) para un mismo doctor.	Se agregó una validación con <code>existsByDoctorIdAndFechaHora(...)</code> antes de guardar, lanzando <code>IllegalArgumentException</code> si ya existe una cita en ese horario.	CitaService.java y CitaRepository.java (nuevo método)
16	Las consultas por doctor y por estado generaban el problema N+1 al cargar el doctor de cada cita por separado.	Se reemplazaron las consultas derivadas por una <code>@Query</code> con <code>JOIN FETCH c.doctor</code> , obteniendo el doctor en una sola consulta.	CitaRepository.java

Tabla 10. Bugs encontrados durante las pruebas en el módulo de citas y su solución.

5.3.3 DoctorControllerIntegrationTest

Se realizaron 33 pruebas:

#	Test	Endpoint	Partición / Caso	Resultado esperado
1	crear_datosValidos_retorna201	POST /api/doctores	Datos válidos	201
2	crear_nombreNulo_retorna400	POST /api/doctores	nombre nulo	400
3	crear_apellidoNulo_retorna400	POST /api/doctores	apellido nulo	400
4	crear_emailNulo_retorna400	POST /api/doctores	email nulo	400
5	crear_emailVacio_retorna400	POST /api/doctores	email vacío	400
6	crear_emailDuplicado_retornaError	POST /api/doctores	email ya registrado	4xx
7	crear_telefonoInvalido_retorna400	POST /api/doctores	teléfono con formato inválido	400
8	crear_nombreConNumeros_retorna400	POST /api/doctores	nombre con números	400



Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas
“Verificación y Validación del Software”

9	crear_nombreVacio_retorna400	POST /api/doctores	nombre en blanco (espacios)	400
10	crear_apellidoVacio_retorna400	POST /api/doctores	apellido en blanco	400
11	crear_especialidadVacia_retorna400	POST /api/doctores	especialidad vacía	400
12	crear_emailInvalido_retorna400	POST /api/doctores	email con formato inválido	400
13	listar_conDatos_retornaLista	GET /api/doctores	Lista con 1 doctor	200, lista tamaño 1
14	listar_sinDatos_retornaListaVacia	GET /api/doctores	Sin doctores registrados	200, lista vacía
15	buscar_idExistente_retornaDoctor	GET /api/doctores/{id}	ID existente	200 con datos del doctor
16	buscar_idInexistente_retorna404	GET /api/doctores/{id}	ID inexistente	404
17	buscar_idNoNumerico_retorna400	GET /api/doctores/{id}	ID no numérico ("abc")	400
18	buscar_idNegativo_retorna404	GET /api/doctores/{id}	ID negativo	404
19	actualizar_idInexistente_retorna404	PUT /api/doctores/{id}	ID inexistente	404
20	actualizar_nombreVacio_retorna400	PUT /api/doctores/{id}	nombre vacío en actualización	400
21	actualizar_emailInvalido_retorna400	PUT /api/doctores/{id}	email inválido en actualización	400
22	actualizar_datosValidos_retornaActualizado	PUT /api/doctores/{id}	Actualización válida	200 con datos actualizados
23	eliminar_idInexistente_retorna404	DELETE /api/doctores/{id}	ID inexistente	404
24	eliminar_idExistente_retorna204	DELETE /api/doctores/{id}	ID existente	204, registro eliminado
25	buscarPorEspecialidad_valorNormal_retornaCoincidencias	GET /api/doctores/buscar-especialidad	Búsqueda normal ("Neuro")	200, ≥ 1 coincidencia
26	buscarPorEspecialidad_payloadSqlInjection_noExplota	GET /api/doctores/buscar-especialidad	Payload SQL Injection	200, 0 resultados.
27	buscarPorEspecialidad_valorVacio_retornaListaVacia	GET /api/doctores/buscar-especialidad	Valor vacío	200, lista vacía
28	buscarPorEspecialidad_sinCoincidencias_retornaListaVacia	GET /api/doctores/buscar-especialidad	Sin coincidencias	200, lista vacía
29	buscarPorEspecialidad_parametroFaltante_retorna400	GET /api/doctores/buscar-especialidad	Parámetro q faltante	400
30	buscarPorNombreCompleto_datosExactos_retornaCoincidencia	GET /api/doctores/buscar-nombre	Nombre y apellido exactos	200, ≥ 1 coincidencia
31	buscarPorNombreCompleto_sinCoincidencia_retornaListaVacia	GET /api/doctores/buscar-nombre	Sin coincidencia	200, lista vacía
32	buscarPorNombreCompleto_parametrosVacios_retorna400	GET /api/doctores/buscar-nombre	Ambos parámetros vacíos	400
33	buscarPorNombreCompleto_faltaParametro_retorna400	GET /api/doctores/buscar-nombre	Falta parámetro apellido	400

Tabla 11. Listado de pruebas de integración del controlador de doctores.

5.3.4 Bugs / Defectos encontrados



Escuela Politécnica Nacional
Facultad de Ingeniería en Sistemas
“Verificación y Validación del Software”

#	Problema detectado	Solución aplicada	Archivo modificado
1	POST /api/doctores retornaba 200 OK en vez de 201 Created.	Se reemplazó ResponseEntity.ok() por ResponseEntity.created(uri), agregando el header Location con la ubicación del recurso creado.	DoctorController.java
2	DELETE /api/doctores/{id} retornaba 200 OK en vez de 204 No Content.	Se reemplazó ResponseEntity.ok().build() por ResponseEntity.noContent().build().	DoctorController.java
3	El endpoint buscar-especialidad llamaba al método inseguro buscarPorEspecialidadInsegura, vulnerable a SQL Injection.	Se cambió la llamada para usar el método seguro y parametrizado buscarPorEspecialidad.	DoctorController.java
4	SQL Injection real: buscarPorEspecialidadInsegura construía la consulta concatenando cadenas directamente con el parámetro de entrada.	Se eliminó la concatenación de cadenas con EntityManager.createNativeQuery(...); ahora usa el repositorio con la consulta derivada parametrizada findByEspecialidadContainingIgnoreCase, que aplica bind de parámetros automáticamente.	DoctorService.java
5	Faltaba @Transactional en los métodos que modifican datos, sin rollback ante fallos.	Se agregó @Transactional(readOnly = true) a nivel de clase y @Transactional específico en crear, actualizar y eliminar.	DoctorService.java
6	eliminar() no verificaba si el doctor existía antes de intentar eliminarlo.	Se agregó existsById() antes de eliminar, lanzando ResourceNotFoundException si no existe.	DoctorService.java
7	eliminar() no validaba si el doctor tenía citas activas antes de eliminarlo.	Se agregó la verificación con citaRepository.existsByDoctorIdAndEstado(id, "PROGRAMADA"), lanzando IllegalArgumentException si tiene citas pendientes.	DoctorService.java y CitaRepository.java (nuevo método existsByDoctorIdAndEstado)
8	buscarPorNombreCompleto no validaba que nombre y apellido no estuvieran vacíos.	Se agregó validación con StringUtils.hasText(...), lanzando IllegalArgumentException si alguno está vacío o contiene solo espacios.	DoctorService.java
9	No se validaba si el email ya estaba registrado por otro doctor al crear uno nuevo.	Se agregó doctorRepository.existsByEmail(dto.getEmail()) antes de guardar, lanzando IllegalArgumentException si ya existe.	DoctorService.java y DoctorRepository.java (nuevo método existsByEmail)
10	No se validaba email duplicado al actualizar un doctor existente.	Se agregó la misma validación en actualizar(), comparando contra el email actual para permitir que el doctor conserve su propio correo sin conflicto.	DoctorService.java
11	El campo especialidad no tenía validaciones, permitiendo valores vacíos.	Se agregó la anotación @NotBlank.	DoctorDTO.java
12	El campo email solo tenía @Email, que no rechaza valores nulos o vacíos.	Se agregó @NotBlank junto con @Email.	DoctorDTO.java
13	El campo telefono no tenía validación de formato.	Se agregaron @NotBlank y @Pattern (solo dígitos, entre 7 y 10 caracteres).	DoctorDTO.java
14	Los campos nombre y apellido aceptaban números y caracteres no alfabéticos.	Se agregó @Pattern que solo permite letras y espacios.	DoctorDTO.java

Tabla 12. Bugs encontrados durante las pruebas en el módulo de doctores y su solución.

5.3.5 HistoriaClínicaControllerIntegrationTest

Se realizaron 20 tests:

#	Test	Endpoint	Partición / Caso	Resultado esperado
1	crear_doctorIdNulo_esValido	POST /api/historias-clinicas	doctorId nulo (opcional)	201, sin campo doctor en la respuesta
2	crear_pacienteIdNulo_retorna400	POST /api/historias-clinicas	pacienteId nulo	400
3	crear_diagnosticoVacio_retorna400	POST /api/historias-clinicas	diagnóstico vacío	400
4	crear_pacienteInexistente_retorna404	POST /api/historias-clinicas	pacienteId inexistente	404



5	crear_doctorInexistente_bugDeteccion	POST /api/historias-clinicas	doctorId inexistente	404
6	crear_contenidoXssEnDiagnostico_bugDeteccion	POST /api/historias-clinicas	Payload XSS en diagnóstico	201, se guarda sin sanitizar
7	crear_diagnosticoNulo_retorna400	POST /api/historias-clinicas	diagnóstico nulo	400
8	crear_diagnosticoExcedeLongitud_retorna400	POST /api/historias-clinicas	diagnóstico excede longitud máxima	400
9	crear_datosValidos_retorna201	POST /api/historias-clinicas	Datos válidos	201
10	buscar_idNoNumerico_retorna400	GET /api/historias-clinicas/{id}	ID no numérico ("abc")	400
11	buscar_idNegativo_retorna404	GET /api/historias-clinicas/{id}	ID negativo	404
12	buscar_idExistente_retornaHistoria	GET /api/historias-clinicas/{id}	ID existente	200 con la historia
13	buscar_idInexistente_retorna404	GET /api/historias-clinicas/{id}	ID inexistente	404
14	listar_variasHistorias_retornaOrdenDescendente	GET /api/historias-clinicas	Varias historias (2)	200, ordenadas por fecha descendente
15	listar_sinHistorias_retornaListaVacía	GET /api/historias-clinicas	Sin historias	200, lista vacía
16	listarPorPaciente_conHistorias_retornaLista	GET /api/historias-clinicas/paciente/{id}	Paciente con historias	200, lista tamaño 1
17	listarPorPaciente_sinHistorias_retornaListaVacía	GET /api/historias-clinicas/paciente/{id}	Paciente sin historias/inexistente	200, lista vacía
18	listarPorDoctor_sinHistorias_retornaListaVacía	GET /api/historias-clinicas/doctor/{id}	Doctor existente sin historias	200, lista vacía
19	listarPorDoctor_doctorInexistente_retornaListaVacía	GET /api/historias-clinicas/doctor/{id}	Doctor inexistente	200, lista vacía
20	listarPorDoctor_conHistorias_retornaLista	GET /api/historias-clinicas/doctor/{id}	Doctor con historias	200, lista tamaño 1

Tabla 13. Listado de pruebas de integración del controlador de historial clínico.

5.3.6 Bugs / Defectos encontrados

#	Problema detectado	Solución aplicada	Archivo modificado
1	POST /api/historias-clinicas retornaba 200 OK en vez de 201 Created.	Se reemplazó ResponseEntity.ok() por ResponseEntity.created(uri), agregando el encabezado Location con la ubicación del recurso creado.	HistoriaClinicaController.java
2	El manejador de ResourceNotFoundException respondía con 200 OK en lugar de 404 Not Found (paciente inexistente).	Se cambió ResponseEntity.ok(body) por ResponseEntity.status(HttpStatus.NOT_FOUND).body(body).	GlobalExceptionHandler.java
3	El manejador de ResourceNotFoundException respondía con 200 OK en lugar de 404 Not Found (ID de historia clínica inexistente).	Se aplicó el mismo cambio global en el manejador de excepciones para devolver 404 Not Found.	GlobalExceptionHandler.java
4	GET /api/historias-clinicas/{id} con ID negativo retornaba 200 OK en vez de 404 Not Found.	Se aprovechó la corrección del GlobalExceptionHandler; ahora ResourceNotFoundException se propaga correctamente como 404 Not Found.	GlobalExceptionHandler.java
5	El campo diagnostico no tenía validación de longitud máxima.	Se agregó la anotación @Size(max = 1000).	HistoriaClinicaDTO.java



6	pacienteId y doctorId aceptaban valores negativos.	Se agregó la anotación @Positive a ambos campos.	HistoriaClinicaDTO.java
7	Los campos tratamiento y observaciones no tenían validación de longitud.	Se agregó la anotación @Size(max = 1000) a ambos campos para mantener consistencia con diagnostico.	HistoriaClinicaDTO.java

Tabla 14. Bugs encontrados durante las pruebas en el módulo de historial y su solución.

5.3.7 PacienteControllerIntegrationTest

Se realizaron 28 pruebas

#	Test	Endpoint	Partición / Caso	Resultado esperado
1	crear_nombreNulo_retorna400	POST /api/pacientes	nombre nulo	400
2	crear_emailNulo_retorna400	POST /api/pacientes	email nulo	400
3	crear_emailDuplicado_retornaError	POST /api/pacientes	email ya registrado	409
4	crear_telefonoConLetras_retorna400	POST /api/pacientes	teléfono con letras	400
5	crear_fechaNacimientoHoy_retorna400	POST /api/pacientes	fecha de nacimiento = hoy (valor límite)	400
6	crear_fechaNacimientoNula_retorna400	POST /api/pacientes	fecha de nacimiento nula	400
7	crear_datosValidos_retorna201	POST /api/pacientes	Datos válidos	201
8	crear_nombreVacio_retorna400	POST /api/pacientes	nombre vacío	400
9	crear_apellidoVacio_retorna400	POST /api/pacientes	apellido vacío	400
10	crear_emailInvalido_retorna400	POST /api/pacientes	email con formato inválido	400
11	crear_telefonoInvalido_retorna400	POST /api/pacientes	teléfono con longitud inválida (5 dígitos)	400
12	crear_telefonoConDiezDigitos_esValido	POST /api/pacientes	teléfono con exactamente 10 dígitos (valor límite válido)	201
13	crear_fechaNacimientoFutura_retorna400	POST /api/pacientes	Fecha de nacimiento futura	400
14	buscar_idExistente_retornaPaciente	GET /api/pacientes/{id}	ID existente	200
15	buscar_idInexistente_retorna404	GET /api/pacientes/{id}	ID inexistente	404
16	buscar_idNoNumerico_retorna400	GET /api/pacientes/{id}	ID no numérico ("abc")	400
17	buscar_idNegativo_retorna404	GET /api/pacientes/{id}	ID negativo	404
18	actualizar_datosValidos_retornaActualizado	PUT /api/pacientes/{id}	Actualización válida	200
19	actualizar_idInexistente_retorna404	PUT /api/pacientes/{id}	ID inexistente	404
20	actualizar_emailInvalido_retorna400	PUT /api/pacientes/{id}	email inválido en actualización	400
21	eliminar_idExistente_retorna204	DELETE /api/pacientes/{id}	ID existente	204
22	eliminar_idInexistente_retorna404	DELETE /api/pacientes/{id}	ID inexistente	404
23	buscarPorNombre_nombreExistente_retornaLista	GET /api/pacientes/buscar	Nombre con coincidencia	200
24	listar_conDatos_retornaLista	GET /api/pacientes	Lista con 1 paciente	200
25	listar_sinDatos_retornaListaVacía	GET /api/pacientes	Sin pacientes registrados	200
26	buscarPorNombre_sinCoincidencias_retornaListaVacía	GET /api/pacientes/buscar	Sin coincidencias	200
27	edadPromedio_conPacientes_calculaCorrectamente	GET /api/pacientes/estadisticas/edad-promedio	Con pacientes (20 y 30 años)	200



28	edadPromedio_sinPacientes_bugDivisionPorCero	GET /api/pacientes/estadisticas/edad-promedio	Sin pacientes	200
----	--	--	---------------	-----

Tabla 15. Listado de pruebas de integración del controlador de paciente.

5.3.8 Bugs / Defectos encontrados

#	Problema detectado	Solución aplicada	Archivo modificado
1	apellido no tenía ninguna validación, permitiendo valores vacíos.	Se agregó @NotBlank.	PacienteDTO.java
2	fechaNacimiento solo tenía @Past (permitía nulo, sin límite de antigüedad).	Se agregó @NotNull junto a @Past.	PacienteDTO.java
3	email solo tenía @Email (permitía nulo/vacío).	Se agregó @NotBlank junto a @Email.	PacienteDTO.java
4	telefono solo tenía @Pattern (permitía nulo).	Se agregó @NotBlank junto a @Pattern.	PacienteDTO.java
5	findByEmail retornaba Paciente directamente (riesgo de NullPointerException si no existía el email).	Se cambió el tipo de retorno a Optional<Paciente>, forzando manejo explícito de ausencia de resultado.	PacienteRepository.java
6	buscarPorEmail no validaba email vacío/nulo y no compilaba tras el cambio del repositorio.	Se agregó validación de email vacío/nulo y se usa .orElseThrow(...) sobre el Optional.	PacienteService.java
7	crear() no validaba si el email ya existía antes de guardar.	Se agregó pacienteRepository.findByEmail(...).isPresent() antes de guardar, lanzando IllegalArgumentException si ya existe.	PacienteService.java
8	El manejador de ResourceNotFoundException respondía con 200 OK en lugar de 404 (afecta buscar_idInexistente, buscar_idNegativo, eliminar_idInexistente, actualizar_idInexistente).	Ya corregido a nivel global: ResponseEntity.status(HttpStatus.NOT_FOUND).body(body).	GlobalExceptionHandler.java
9	POST /api/pacientes retornaba 200 OK en vez de 201 Created.	Se aplicó ResponseEntity.created(uri) con header Location.	PacienteController.java
10	DELETE /api/pacientes/{id} retornaba 200 OK en vez de 204 No Content.	Se aplicó ResponseEntity.noContent().build().	PacienteController.java
11	ID no numérico en GET /{id} generaba error no controlado.	Ya corregido a nivel global con el handler de MethodArgumentTypeMismatchException.	GlobalExceptionHandler.java

Table 16. Bugs encontrados durante las pruebas en el módulo de paciente y su solución.

Bugs pendientes (Hallazgos)

#	Problema detectado	Estado
12	buscarPorNombre usa una consulta nativa vulnerable a SQL Injection (concatenación con) y falla con NullPointerException si el parámetro es nulo.	Pendiente. Requiere reemplazar por query derivada parametrizada (findByNombreContainingIgnoreCase) y validar nulo antes de consultar.
13	calcularEdadPromedio() divide por cero si no hay pacientes registrados.	Pendiente (documentado en edadPromedio_sinPacientes_bugDivisionPorCero). Requiere validar pacientes.isEmpty() antes de dividir.
14	eliminar() hace borrado físico sin verificar si el paciente tiene citas o historias clínicas asociadas.	Pendiente. Requiere validación de dependencias, similar al fix aplicado en DoctorService.eliminar().
15	@CrossOrigin(origins = "**") a nivel de clase, CORS demasiado permisivo.	Pendiente. Requiere origen configurable, igual que en CitaController.

Resultados de la Implementación



Antes:

Figura 1. Resultado de pruebas de integración del módulo citas antes de la corrección de bugs.

Figura 2. Resultado de pruebas de integración del módulo doctores antes de la corrección de bugs.

Figura 3. Resultado de pruebas de integración del módulo historial clínico antes de la corrección de bugs.

Figura 4. Resultado de pruebas de integración del módulo paciente antes de la corrección de bugs.

Después:

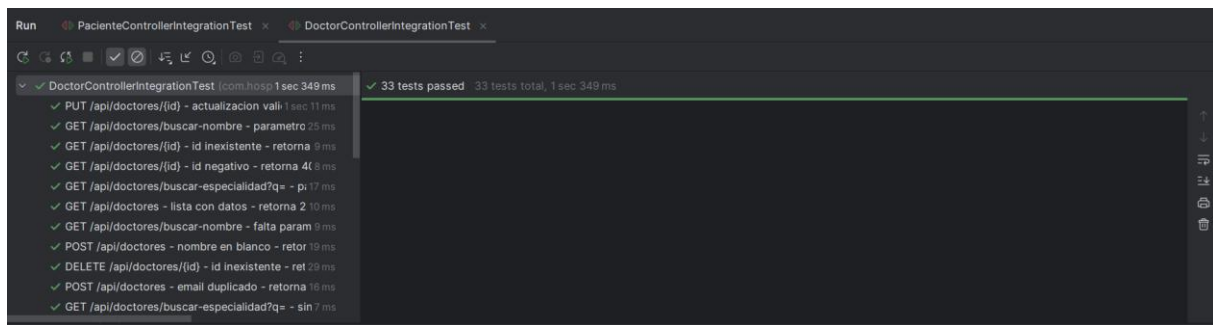


Figure 5. Resultado de pruebas de integración del módulo doctores después de la corrección de bugs.

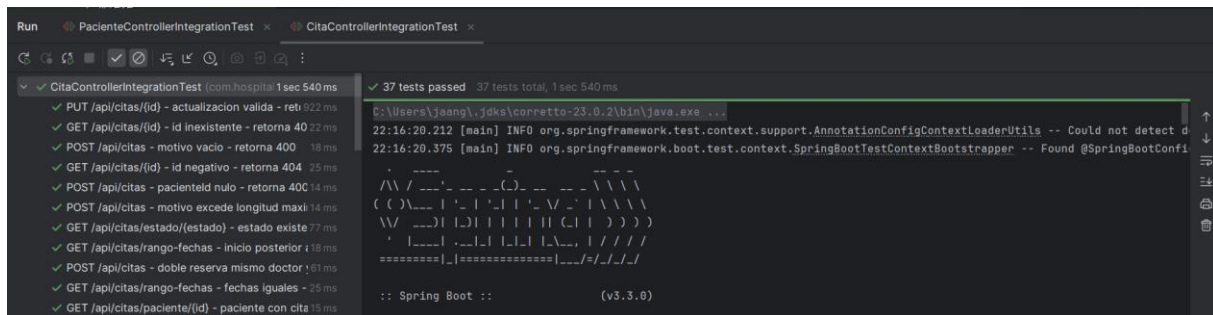


Figura 6. Resultado de pruebas de integración del módulo citas después de la corrección de bugs.

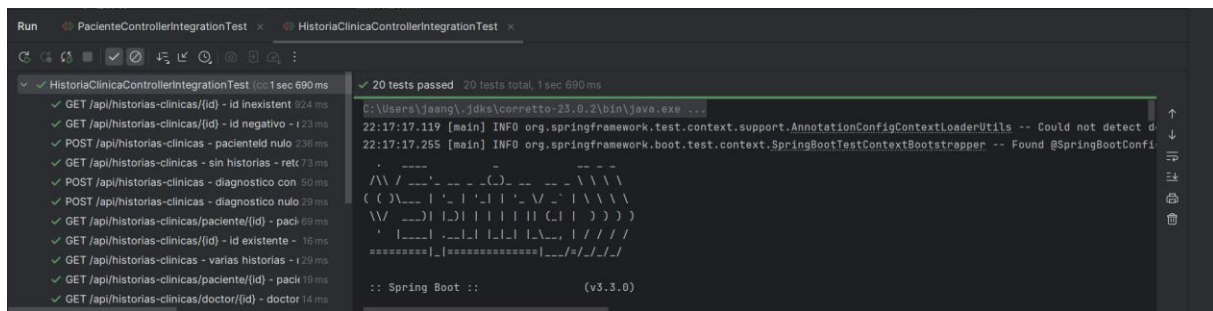


Figura 7. Resultado de pruebas de integración del módulo historial clínico después de la corrección de bugs.

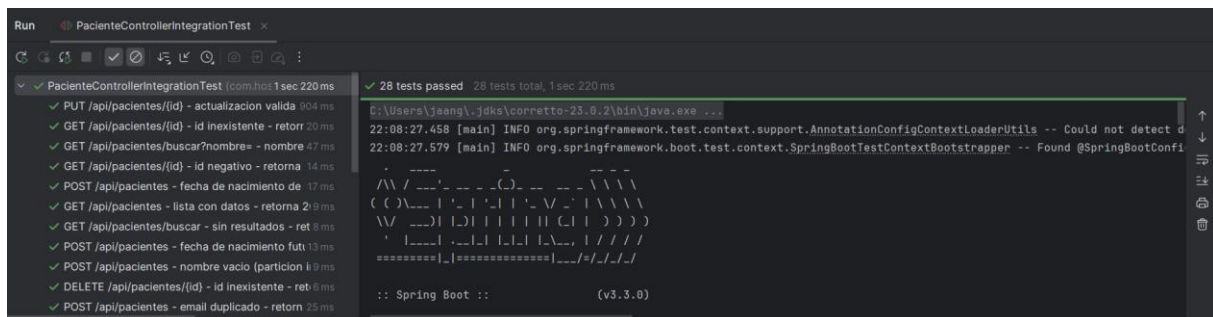


Figure 8. Resultado de pruebas de integración del módulo paciente después de la corrección de bugs.

6. Conclusiones

- A lo largo de este trabajo se implementaron y ejecutaron 129 pruebas de integración sobre los cuatro controladores principales del backend del sistema Hospital Management (Cita, Doctor, HistoriaClínica y Paciente), utilizando `@SpringBootTest` y `MockMvc` para simular peticiones HTTP reales sobre la aplicación completa. Este proceso permitió identificar y corregir defectos significativos que no eran evidentes en una revisión superficial del código.



- La aplicación sistemática de particionamiento de equivalencia y valores límite en cada endpoint permitió cubrir tanto los casos de uso esperados como los escenarios de error, demostrando que las pruebas de integración no solo verifican que el sistema funcione, sino que también actúan como una herramienta efectiva para descubrir defectos ocultos en la lógica de negocio y en el manejo de excepciones a nivel global.